BALANÇO FINAL DE IMPLEMENTAÇÃO • TCC ENGENHARIA DE SOFTWARE / COMPUTAÇÃO

ReqFlow — Comparativo Projeto TCC × Sistema Implementado

O que está 100% Concluído, o que foi Adaptado/Simplificado e o que foi Retirado do Escopo

TOTAL ANALISADO	IMPLEMENTADOS	ADAPTADOS (ÁGIL)	RETIRADOS DO ESCOPO
44 RF + 11 RNF	**38 RF / 11 RNF**	**2 RF (RF.11, RF.14)**	**4 RF (19, 29, 30, 35)**
55 itens na planilha inicial	100% do escopo ativo entregue	Foco em 3 status + RF/RNF	Cortados por decisão do TCC

1. Resumo das Decisões de Escopo: O que Retiramos e o que Adaptamos

ID	Requisito Original na Planilha	Decisão	Justificativa Técnica / Metodológica para o Texto do TCC
RF.19	Refinar requisitos antes da aprovação, sendo vedada a alteração após aprovação	**Retirado**	Retirado por decisão do autor: no desenvolvimento ágil, requisitos precisam evoluir continuamente mesmo após aprovados.
RF.29	Cadastrar casos de uso com atores, fluxos, pré-condições e pós-condições	**Retirado**	Substituído pelo fluxo ágil de User Stories (Como/Quero/Para) + Critérios de Aceitação, evitando duplicidade com UML.
RF.30	Vincular requisitos funcionais aos casos de uso correspondentes	**Retirado**	Retirado em decorrência da exclusão do módulo de Casos de Uso (RF.29); a rastreabilidade ocorre com User Stories (RF.16).
RF.35	Anexar documentos, imagens e arquivos aos requisitos	**Retirado**	Retirado para manter o foco na especificação textual estruturada (padrão IEEE 830) sem armazenamento binário externo.
RF.14	Acompanhar o status de cada requisito	**Adaptado**	Simplificado para 3 status de homologação (Em Análise, Aprovado, Não Aprovado). O ciclo de dev fica nas User Stories/Sprints.
RF.11	Classificar requisitos por tipo	**Adaptado**	Focado na dupla clássica RF (Funcional) e RNF (Não-Funcional), mantendo a interface enxuta e direta.

2. Auditoria Completa dos Requisitos Funcionais (RF.01 a RF.44)

2.1 Autenticação, Usuários e Controle de Permissões por Projeto (RF.01 – RF.05)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.01	Permitir o cadastro de usuários	**Implementado**	Tela register/page.jsx integrada ao Supabase Auth; o usuário cria a conta sem escolher função global (definida por projeto).	Concluído
RF.02	Permitir login e logout	**Implementado**	Login em login/page.jsx, proteção SSR em middleware.js e logout na Sidebar e no menu superior.	Concluído
RF.03	Permitir a edição de perfil do usuário	**Implementado**	Modal 'Editar Meu Perfil' (edit-profile-modal.jsx) acessível na Topbar para atualizar nome, avatar e senha.	Concluído
RF.04	Definir perfis de acesso no projeto	**Implementado**	6 funções padrão por projeto (Admin, Analista, PO, Scrum Master, Dev, Gerente) + criação de funções customizadas.	Concluído
RF.05	Controlar permissões conforme o perfil do usuário	**Implementado**	Aba 'Funções & Permissões' com 12 checkboxes por projeto, proteção Server Action, botões semi-transparentes e toast.	Concluído

2.2 Gestão de Projetos, Convites e Stakeholders (RF.06 – RF.08, RF.26)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.06	Gerenciar projetos de software (cadastrar, editar e excluir)	**Implementado**	Criação de projeto na Topbar, edição de dados gerais e Zona de Perigo para Exclusão do Projeto em manage-project-modal.jsx.	Concluído
RF.07	Listar os projetos vinculados ao usuário	**Implementado**	Isolamento estrito por usuário: lista apenas projetos que o usuário criou ou nos quais aceitou convite.	Concluído
RF.08	Permitir o gerenciamento de um projeto por mais de um usuário	**Implementado**	Múltiplos membros em project_members, convites por e-mail (sem expor lista de usuários), aceite/recusa e opção de sair.	Concluído
RF.26	Gerenciar stakeholders do projeto (convidar, editar função e remover)	**Implementado**	Aba 'Stakeholders do Projeto' com convite por e-mail, alteração de papel restrita a Admin e trava de auto-alteração.	Concluído

2.3 Especificação, Aprovação e Qualidade de Requisitos (RF.10 – RF.15, RF.18, RF.19, RF.25, RF.32, RF.36)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.10	Gerenciar requisitos funcionais e não funcionais vinculados ao projeto (CRUD)	**Implementado**	Cadastro, listagem, edição e exclusão de requisitos em requisitos/page.jsx e requirement-modal.jsx.	Concluído
RF.11	Classificar requisitos por tipo	**Adaptado**	Classifica em RF (Requisito Funcional) e RNF (Requisito Não-Funcional) com filtros e badges dedicados.	Foco em RF e RNF
RF.12	Gerar identificador único para cada requisito (ex: REQ-001, RF-G01)	**Implementado**	Gera código automático único por projeto ou biblioteca global e valida duplicidade.	Concluído
RF.13	Definir prioridade de cada requisito (baixa, média, alta ou crítica)	**Implementado**	Suporta as 4 prioridades com badges semânticos e filtros dedicados.	Concluído
RF.14	Acompanhar o status de cada requisito	**Adaptado**	Simplificado para 3 status de homologação: Em Análise, Aprovado e Não Aprovado (implementação ocorre nas User Stories).	Adaptado (3 status)
RF.15	Registrar descrição detalhada, justificativa, fonte e critério de aceitação	**Implementado**	Todos os campos estruturados no requirement-modal.jsx e incluídos na geração do documento SRS.	Concluído
RF.18	Registrar a fonte/origem do requisito	**Implementado**	Campo Fonte/Origem salvo no requisito e validado pelo motor de completude.	Concluído
RF.19	Refinar requisitos antes da aprovação, sendo vedada alteração após aprovado	**Retirado**	Não aplicável: edição permanece aberta após aprovação para suportar evolução ágil contínua.	Retirado pelo autor
RF.25	Validar/aprovar requisitos por funções com permissão	**Implementado**	Botões de 1 clique (Aprovar / Não Aprovar) direto na tabela de requisitos, controlados pela permissão requirements_approve.	Concluído
RF.32	Consultar e filtrar por código, tipo, prioridade, status, projeto ou texto	**Implementado**	Busca em tempo real e filtros combinados por Tipo, Prioridade e Status em requisitos/page.jsx.	Concluído
RF.36	Alertar quando um requisito estiver incompleto	**Implementado**	Trigger check_requirement_completeness no banco e alerta visual (AlertTriangle) na tabela e nos relatórios.	Concluído

2.4 Metodologia Ágil: User Stories, Backlog e Sprints (RF.09, RF.16, RF.17, RF.27, RF.28)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.09	Gerenciar histórias de usuário no formato 'Como/Quero/Para' (CRUD completo)	**Implementado**	Criação, edição, exclusão e visualização de User Stories em user-stories/page.jsx com controle de permissão.	Concluído
RF.16	Associar requisitos às histórias de usuário cadastradas no projeto	**Implementado**	Vínculo N:N (requirement_user_stories) disponível no modal de User Story, no modal de Requisito e na Matriz.	Concluído
RF.17	Associar requisitos a stakeholders responsáveis	**Implementado**	Seleção de responsável no modal de Requisito e vínculo multi-stakeholder na Matriz de Rastreabilidade.	Concluído
RF.27	Organizar histórias de usuário em um backlog priorizado	**Implementado**	Quadro Kanban em backlog/page.jsx com Product Backlog, reordenação e prioridades.	Concluído
RF.28	Vincular histórias de usuário a sprints de desenvolvimento	**Implementado**	Gestão completa de Sprints (criar, ativar, concluir, arquivar, excluir) e alocação de User Stories por Sprint.	Concluído

2.5 Rastreabilidade, Dependências, Análise de Impacto e Casos de Uso (RF.20, RF.21, RF.29 – RF.31)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.20	Registrar dependências entre requisitos (conflito, complemento, refinamento)	**Implementado**	Aba 'Impacto' no modal do requisito permite vincular e excluir dependências (depends_on, complements, refines, conflict).	Concluído
RF.21	Realizar análise de impacto identificando requisitos e artefatos afetados	**Implementado**	Banner automático de Análise de Impacto ao editar requisito listando requisitos dependentes e User Stories afetadas.	Concluído
RF.29	Cadastrar casos de uso com atores, fluxos, pré e pós-condições	**Retirado**	Fora do escopo: o sistema utiliza User Stories e Critérios de Aceitação como artefato funcional ágil.	Retirado do escopo
RF.30	Vincular requisitos funcionais aos casos de uso correspondentes	**Retirado**	Fora do escopo (substituído pelo vínculo Requisito <-> User Story no RF.16).	Retirado do escopo
RF.31	Gerar matriz de rastreabilidade interativa (Requisitos × US × Stakeholders)	**Implementado**	Matriz bidirecional em rastreabilidade/page.jsx com persistência imediata no Supabase ao clicar nas células e exportação CSV.	Concluído

2.6 Versionamento, Decisões, Reuso, Notificações e Auditoria (RF.22 – RF.24, RF.33 – RF.35, RF.41 – RF.44)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.22	Manter histórico de alterações registrando data e usuário responsável	**Implementado**	Tabela requirement_versions registra autor, data/hora, resumo e snapshot JSONB a cada edição.	Concluído
RF.23	Controlar versões possibilitando comparar versões anteriores e atuais	**Implementado**	Inspeção/comparação de qualquer versão anterior na timeline do modal com botão 'Restaurar esta Versão'.	Concluído
RF.24	Registrar decisões e comentários vinculados aos requisitos	**Implementado**	Aba 'Decisões' no modal do requisito permite publicar Comentários Técnicos e Decisões Formais de Requisito.	Concluído

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.33	Reutilizar requisitos de outros projetos	**Implementado**	Modal Importar/Reutilizar (import-requirements-modal.jsx) realiza cópia profunda (Deep Clone) entre projetos.	Concluído
RF.34	Manter uma biblioteca de requisitos genéricos e reutilizáveis	**Implementado**	Página '/requisitos/globais' exclusiva por usuário + catálogo de templates prontos para importação rápida.	Concluído
RF.35	Anexar documentos, imagens e arquivos aos requisitos	**Retirado**	Fora do escopo: especificação focada em documentação estruturada SRS IEEE 830 sem upload binário.	Retirado do escopo
RF.41	Notificar usuários sobre alterações, comentários, aprovações ou pendências	**Implementado**	Sino de notificações na Topbar exibe convites pendentes de projetos e alertas das últimas ações do projeto ativo.	Concluído
RF.42	Manter trilha de auditoria registrando as principais ações dos usuários	**Implementado**	Tabela audit_logs registra criações, edições, aprovações, decisões, convites e mudanças de funções, exibidos no Dashboard.	Concluído
RF.43	Pesquisar requisitos semelhantes para evitar duplicidades	**Implementado**	Detecção semântica em tempo real no modal do requisito que alerta quando há requisito com título/descrição similar.	Concluído
RF.44	Apresentar a linha do tempo de evolução de cada requisito	**Implementado**	Linha do tempo vertical interativa na coluna direita do requirement-modal.jsx.	Concluído

2.7 Dashboard, Relatórios Analíticos e Exportação SRS (RF.37 – RF.40)

ID	Requisito na Tabela	Situação	Como está implementado no Sistema (ReqFlow)	Observação / Pendência
RF.37	Gerar automaticamente documento de especificação de requisitos (SRS)	**Implementado**	Gerador SRS no padrão IEEE 830 / ISO 29148 em relatorios/page.jsx com impressão nativa para PDF e download .TXT.	Concluído
RF.38	Exportar requisitos em diferentes formatos (PDF, CSV, JSON, TXT)	**Implementado**	Exportação real em .CSV (compatível com Excel UTF-8 BOM), .JSON estruturado, .TXT e .PDF em relatorios/page.jsx.	Concluído
RF.39	Apresentar dashboard com indicadores do projeto	**Implementado**	Dashboard reformulado com KPIs dos 3 status (Em Análise, Aprovados, Não Aprovados), progresso de User Stories e Donut Chart.	Concluído
RF.40	Gerar relatórios por tipo, status, prioridade, completude e responsável	**Implementado**	Painéis analíticos detalhados na aba 'Exportações & Relatórios Analíticos' em relatorios/page.jsx.	Concluído

3. Situação Final dos Requisitos Não-Funcionais (RNF.01 a RNF.11)

ID	Descrição do RNF na Tabela	Situação	Evidência Técnica no Sistema Atual
RNF.01	Possuir interface simples e intuitiva	**Implementado**	Interface em Tailwind CSS v4 com Modo Claro/Escuro, contraste aprimorado e ícones Lucide.
RNF.02	Ser acessível via navegador web	**Implementado**	Aplicação web moderna em Next.js App Router pronta para produção na Vercel.
RNF.03	Garantir autenticação segura de usuários	**Implementado**	Supabase Auth (JWT + SSR Cookies) e proteção de rotas em src/middleware.js.
RNF.04	Proteger dados contra acesso não autorizado	**Implementado**	Isolamento por projeto/usuário, validação em Server Actions e Matriz de Permissões por função.
RNF.05	Ser desenvolvido como aplicação web	**Implementado**	Arquitetura full-stack web com React 19 e Next.js 16.
RNF.06	Utilizar banco de dados para persistência	**Implementado**	Banco relacional PostgreSQL no Supabase com 14 tabelas e migrations versionadas.
RNF.07	Estar disponível em diferentes dispositivos e resoluções	**Implementado**	Grids responsivos e Sidebar retrátil com drawer flutuante para telas mobile e tablets.
RNF.08	Responder às requisições em tempo adequado	**Implementado**	Atualizações otimistas na UI e consultas indexadas no PostgreSQL.
RNF.09	Ser de fácil manutenção e expansão	**Implementado**	Código modularizado em Server Actions, Context Providers, componentes de domínio e constantes.
RNF.10	Registrar logs e auditoria para rastreamento	**Implementado**	Tratamento try/catch em todas as Server Actions com feedback visual e registro em audit_logs.
RNF.11	Garantir a integridade dos dados armazenados	**Implementado**	Chaves estrangeiras, ON DELETE CASCADE, constraints de unicidade e triggers PL/pgSQL.